

Mini-Projet 1

Pipeline ETL & Data Warehouse

Mexora Analytics — E-commerce marocain

Réalisé par	Soufiane Zaari
Encadrant	Prof. Hassan Zili
Année	2025 – 2026
GitHub	SoufianeZaari/Pipeline-ETL-Data-Warehouse
Dashboard web	pipeline-etl-data-warehouse.streamlit.app

Étape	Poids	Statut
Étape 1 — Modélisation du Data Warehouse	25 %	Conforme
Étape 2 — Pipeline ETL Python	40 %	Conforme
Étape 3 — Implémentation PostgreSQL	20 %	Conforme
Étape 4 — Dashboard BI / Streamlit	15 %	Conforme

Résultats clés — Pipeline validé

Commandes brutes	50 000 lignes	Vues matérialisées	3 vues
Faits chargés	36 203 lignes	Durée pipeline	14 secondes
Dimensions	5 tables	Anomalies	0 critique

0 Table des matières

Résumé exécutif	iii
1 Étape 1 — Modélisation du Data Warehouse (25 pts)	1
1.1 Analyse des besoins décisionnels — Requêtes-types	1
1.2 Schéma en étoile — FAIT_VENTES	1
1.2.1 Granularité justifiée	1
1.2.2 Mesures et additivité	2
1.2.3 Cinq dimensions	2
1.3 Slowly Changing Dimensions (SCD)	2
1.3.1 Cas 1 — DIM_PRODUIT : changement de catégorie (SCD Type 2)	2
1.3.2 Cas 2 — DIM_CLIENT : changement de segment (SCD Type 2) .	3
2 Étape 2 — Pipeline ETL en Python (40 pts)	4
2.1 Structure du package <code>mexora_etl/</code>	4
2.2 Phase Extract	4
2.3 Phase Transform — Commandes (7 règles obligatoires)	5
2.4 Phase Transform — Clients (5 règles)	6
2.5 Phase Transform — Produits	6
2.6 Construction des dimensions	6
2.7 Phase Load — PostgreSQL	7
2.8 Orchestration — <code>main.py</code>	8
3 Étape 3 — Implémentation PostgreSQL (20 pts)	10
3.1 Architecture en trois schémas	10
3.2 Dimensions et table de faits	10
3.3 Indexation	11
3.4 Trois vues matérialisées	12
3.4.1 Vue 1 — <code>mv_ca_mensuel</code> (914 lignes)	12
3.4.2 Vue 2 — <code>mv_top_produits</code> (3013 lignes)	12
3.4.3 Vue 3 — <code>mv_performance_livresseurs</code> (215 lignes)	12
3.5 Vérification d'intégrité référentielle	13
4 Étape 4 — Dashboard de visualisation (15 pts)	14
4.1 Questions analytiques obligatoires	14
4.1.1 Q1 — Évolution du CA par région (sur 12 mois, comparaison N-1)	14
4.1.2 Q2 — Top 10 produits trimestriels à Tanger	14
4.1.3 Q3 — Panier moyen par segment client	14
4.1.4 Q4 — Taux de retour par catégorie (avec alertes)	15
4.1.5 Q5 — Effet Ramadan sur les ventes alimentaires	15

4.2	Version web déployable — Streamlit	16
4.3	Synthèse des insights métier	17
5	Conclusion	18

0 Résumé exécutif

Ce projet implémente un **système décisionnel complet** pour **Mexora Analytics**, une marketplace e-commerce fictive basée à Tanger (Maroc). Il répond au cahier de charge du mini-projet 1 en couvrant les quatre étapes évaluées : modélisation, pipeline ETL, Data Warehouse PostgreSQL et dashboard analytique.

Périmètre et résultats en un coup d'œil

Données source	4 fichiers bruts intentionnellement défectueux : 50 000 commandes, 1 000 clients, 300 produits, référentiel géographique
Pipeline ETL	Package <code>mexora_etl/</code> — 14 règles de transformation, logging complet, 14 s d'exécution
Data Warehouse	PostgreSQL 16, schéma en étoile (1 table de faits, 5 dimensions), 36 203 faits, 3 vues matérialisées, 0 anomalie d'intégrité
Dashboard BI	Metabase local (conforme CDC) + Streamlit Cloud (version web déployable)

Principaux indicateurs métier issus du DWH : CA total 45,6 M MAD, taux de retour 7,87 % (seuil d'alerte 5 %), délai moyen de livraison 14,99 jours, top région Tanger (17,4 % du CA).

1 Étape 1 — Modélisation du Data Warehouse (25 pts)

1.1

Avant toute modélisation, les besoins analytiques du Directeur Général de Mexora ont été formalisés selon la **méthode Top-Down** en six requêtes-types (cinq minimales requises par le cahier de charge).

Réf	Analyser	En fonction de	Pour (filtre)
R1	Chiffre d'affaires TTC	Région, catégorie produit, trimestre	Statut <i>livré</i> uniquement
R2	Quantité vendue et montant TTC	Produit, mois, segment client	Années 2024–2025
R3	Panier moyen par commande	Segment Gold / Silver / Bronze, région	Clients actifs dans le DWH
R4	Taux de retour (%)	Catégorie de produit, région administrative	Toutes les lignes de vente
R5	Volume vendu alimentaire	Période Ramadan vs hors Ramadan, mois	Catégorie Alimentation, 2022–2026
R6	Délai moyen de livraison	Livreur, zone de couverture, mois	Commandes livrées ou retournées

Table 1 – Requêtes-types formalisées — méthode Top-Down

1.2

1.2.1

Granularité choisie : une ligne

Une ligne de **FAIT_VENTES** représente **une commande individuelle** : un produit précis acheté par un client précis, livré dans une ville précise, à une date précise. Cette granularité permet d’agréger le CA sur toutes les dimensions (produit, client, région, période) et de calculer le taux de retour par produit ou catégorie.

1.2.2

Mesure	Additivité	Justification
<code>quantite_vendue</code>	Additive	Sommable par produit, région, période
<code>montant_ht</code>	Additive	CA hors TVA, sommable sur toutes les dimensions
<code>montant_ttc</code>	Additive	CA TTC (TVA 20 %), sommable
<code>delai_livraison_jours</code>	Semi-additive	Agréger par moyenne, jamais par somme
<code>remise_pct</code>	Non-additive	Taux à recalculer selon le contexte
Taux de retour	Non-additive	SUM(retours)/COUNT(*)
Panier moyen	Non-additive	SUM(ttc)/COUNT(DISTINCT id_vente)

Table 2 – Mesures de FAIT_VENTES — types d’additivité

1.2.3

Dimension	Attributs (conformes au CDC)
DIM_TEMPS	<code>id_date</code> , <code>jour</code> , <code>mois</code> , <code>trimestre</code> , <code>annee</code> , <code>libelle_mois</code> , <code>est_weekend</code> , <code>est_ferie_maroc</code> , <code>periode_ramadan</code>
DIM_PRODUIT	<code>id_produit_sk/nk</code> , <code>nom_produit</code> , <code>categorie</code> , <code>sous_categorie</code> , <code>marque</code> , <code>fournisseur</code> , <code>prix_standard</code> , <code>date_debut</code> , <code>date_fin</code> , <code>est_actif</code>
DIM_CLIENT	<code>id_client_sk/nk</code> , <code>nom_complet</code> , <code>tranche_age</code> , <code>sexe</code> , <code>ville</code> , <code>region_admin</code> , <code>segment_client</code> , <code>canal_acquisition</code> , <code>date_debut</code> , <code>date_fin</code> , <code>est_actif</code>
DIM_REGION	<code>id_region</code> , <code>ville</code> , <code>province</code> , <code>region_admin</code> , <code>zone_geo</code> , <code>pays</code>
DIM_LIVREUR	<code>id_livreur</code> , <code>id_livreur_nk</code> , <code>nom_livreur</code> , <code>type_transport</code> , <code>zone_couverture</code>

Table 3 – Cinq dimensions du schéma en étoile (conformes au CDC)

1.3

1.3.1

Un produit peut migrer de *Téléphones* vers *Smartphones* après une refonte du catalogue fournisseur. Le projet **modélise et prépare** le SCD Type 2 : les colonnes `date_debut`, `date_fin`, `est_actif` (+ contrainte `UNIQUE(id_produit_nk, date_debut)`) rendent possible la conservation de l’historique analytique. Les ventes antérieures restent liées à l’ancienne version du produit, garantissant la cohérence temporelle.

Les produits avec `"actif": false` dans la source JSON sont conservés dans `dim_produit` avec `est_actif = FALSE` pour ne pas casser les jointures historiques.

1.3.2

Un client peut passer de *Bronze* à *Silver* lorsque son CA cumulé des 12 derniers mois franchit 5 000 MAD. La structure du DWH est **compatible SCD Type 2** sur `dim_client` : les colonnes d'historisation permettraient, dans une évolution future, de savoir à quel segment un client appartenait lors de chaque achat passé.

Distinction SCD Type 1 / Type 2 — choix motivés

SCD Type 1 pour les *corrections de qualité* (villes mal orthographiées, emails invalides) : ce sont des erreurs sans valeur historique — écraser la valeur est correct.

SCD Type 2 (modélisé) pour les *changements métier réels* (catégorie produit, segment client) : la traçabilité analytique est critique. Le schéma inclut les colonnes d'historisation ; l'activation d'un mécanisme de versionnement automatique est documentée comme évolution naturelle du Data Warehouse.

2 Étape 2 — Pipeline ETL en Python (40 pts)

2.1

```

1  mexora_etl/
2  |-- config/
3  |   '-- settings.py          # chemins, URLs MySQL et PostgreSQL,
   get_pg_engine()
4  |-- extract/
5  |   '-- extractor.py         # extract_commandes, _produits,
   _clients, _regions
6  |-- transform/
7  |   |-- clean_commandes.py   # 7 regles obligatoires R1-R7
8  |   |-- clean_clients.py     # 5 regles R1-R5
9  |   |-- clean_produits.py    # normalisation categories, prix, SCD
10 |   '-- build_dimensions.py#
   build_dim_temps/client/produit/region/livreur
11 |                               # build_fait_ventes,
   calculer_segments_clients
12 |-- load/
13 |   '-- loader.py            # charger_dimension
   (TRUNCATE+INSERT), charger_faits
14 |-- utils/
15 |   '-- logger.py            # configuration logging ETL
16 '-- main.py                  # orchestration E -> T -> L
   (PostgreSQL)

```

Listing 1 – Arborescence du package Python (structure conforme au CDC)

2.2

Toutes les sources sont lues en `dtype=str` pour éviter toute conversion implicite de pandas avant nettoyage :

```

1  def extract_commandes(filepath: str) -> pd.DataFrame:
2      df = pd.read_csv(filepath, encoding='utf-8', dtype=str)
3      logging.info("[EXTRACT] Commandes: %s lignes", len(df))
4      return df
5
6  def extract_produits(filepath: str) -> pd.DataFrame:
7      with open(filepath, 'r', encoding='utf-8') as f:
8          data = json.load(f)
9      df = pd.DataFrame(data['produits'])
10     logging.info("[EXTRACT] Produits: %s lignes", len(df))
11     return df

```

Listing 2 — Fonctions d'extraction — extractor.py

2.3

Règle	Description	Traitement appliqué	Impact
R1	Doublons <code>id_commande</code>	<code>drop_duplicates(keep='last')</code>	−1 500 lignes
R2	Dates en formats mixtes	<code>to_datetime(format='mixed')</code>	0 invalide
R3	Villes incohérentes	Mapping via référentiel <code>regions_maroc</code>	Standardisées
R4	Statuts non standards	OK→ <code>en_cours</code> , KO→ <code>annulé</code> , DONE→ <code>livré</code>	6 812 mappés
R5	Quantités ≤ 0	Suppression de la ligne	−390 lignes
R6	Prix unitaire = 0	Suppression (commandes test)	−1 285 lignes
R7	Livreurs manquants	Remplacement par <code>-1</code> (inconnu)	6 570 imputés
Résultat : 50 000 → 46 825 lignes → 36 203 faits après jointures			−3 175

Table 4 — Règles de transformation — commandes_mexora.csv

```

1 def transform_commandes(df: pd.DataFrame, regions:
  pd.DataFrame) -> pd.DataFrame:
2     initial = len(df)
3     # R1 - Doublons sur id_commande
4     before = len(df)
5     df = df.drop_duplicates(subset=['id_commande'], keep='last')
6     logging.info("[TRANSFORM] R1 doublons: %s lignes
       supprimées", before - len(df))
7     # R2 - Standardisation des dates (formats mixtes)
8     df['date_commande'] = pd.to_datetime(
9         df['date_commande'], format='mixed', dayfirst=True,
        errors='coerce')
10    invalid = int(df['date_commande'].isna().sum())
11    df = df.dropna(subset=['date_commande'])
12    logging.info("[TRANSFORM] R2 dates: %s invalides
       supprimees", invalid)
13    # R3 - Harmonisation des villes via le référentiel
14    mapping_villes = charger_referentiel_villes(regions)
15    df['ville_livraison'] =
        (df['ville_livraison'].str.strip().str.lower())

```

```

16         .map(mapping_villes).fillna('Non
17         renseignee'))
return df

```

Listing 3 – Extrait transform_commandes.py — R1 & R2

2.4

Règle	Description	Traitement	Impact
R1	Doublons email	Garder l'inscription la plus récente	−102 clients
R2	Sexe hétérogène	m/f/1/0/Homme/Femme → m/f/?	Standardisé
R3	Âges invalides	Invalidiser si age < 16 ou > 100 ans	Nullifiés
R4	Emails mal formatés	Validation regex RFC 5322	47 invalides
R5	Segmentation client	Calcul Gold/Silver/Bronze depuis CA 12 mois	Enrichissement
Résultat : 1 000 → 898 clients			−102

Table 5 – Règles de transformation — clients_mexora.csv

2.5

Règle	Traitement	Impact
Catégories incohérentes	electronique / ELECTRONIQUE → Electronique	Corrigé
Prix catalogue nuls	Imputation par médiane de catégorie, sinon 100 MAD	13 corrigés
Produits inactifs	Conservés avec <code>est_actif=False</code> (SCD)	11 détectés
Colonnes SCD Type 2	Ajout <code>date_debut</code> , <code>date_fin</code> , <code>est_actif</code>	300 lignes

Table 6 – Règles de transformation — produits_mexora.json

2.6

```

1 def build_dim_temps(date_debut: str, date_fin: str) ->
  pd.DataFrame:
2     dates = pd.date_range(start=date_debut, end=date_fin,
3     freq='D')
4     ferries_maroc = {
5         '2024-01-01', # Nouvel An
6         '2024-01-11', # Manifeste de l'Indépendance
7         '2024-05-01', # Fete du Travail

```

```

7      '2024-07-30', # Fete du Trone
8      '2024-11-18', # Fete de l'Independance
9      # + 2025-2026 ...
10     }
11     ramadan_periods = [
12         ('2024-03-10', '2024-04-09'),
13         ('2025-03-01', '2025-03-30'),
14         ('2026-02-18', '2026-03-19'),
15     ]
16     df['est_ferie_maroc'] =
17         dates.strftime('%Y-%m-%d').isin(feries_maroc)
18     df['periode_ramadan'] = False
19     for debut, fin in ramadan_periods:
20         masque = (df['date_complete'] >= debut) &
21             (df['date_complete'] <= fin)
22         df.loc[masque, 'periode_ramadan'] = True
23     return df # 2 557 lignes (2020-01-01 a 2026-12-31)

```

Listing 4 — build_dim_temps.py — fériés marocains et Ramadan

```

1 def calculer_segments_clients(commandes: pd.DataFrame) ->
2     pd.DataFrame:
3     """
4     Seuils Mexora :
5     Gold : CA 12 mois >= 15 000 MAD
6     Silver : CA 12 mois >= 5 000 MAD
7     Bronze : CA 12 mois < 5 000 MAD
8     """
9     commandes['montant_ttc'] =
10         (commandes['quantite'].astype(float)
11             *
12             commandes['prix_unitaire'].astype(float))
13     ca =
14         commandes.groupby('id_client')['montant_ttc'].sum().reset_index()
15     ca['segment_client'] = pd.cut(ca['montant_ttc'],
16         bins=[-1, 4999.99, 14999.99, float('inf')],
17         labels=['Bronze', 'Silver', 'Gold']
18     ).astype(str)
19     return ca

```

Listing 5 — calculer_segments_clients.py — Gold / Silver / Bronze

2.7

```

1 def charger_dimension(df, table_name, engine,
2     if_exists='replace'):

```

```

2      """TRUNCATE CASCADE + INSERT pour respecter les FK
        PostgreSQL."""
3  with engine.connect() as conn:
4      conn.execute(text(
5          f"TRUNCATE TABLE dwh_mexora.{table_name} RESTART
            IDENTITY CASCADE"
6      ))
7      conn.commit()
8  df.to_sql(table_name, engine, schema='dwh_mexora',
9          if_exists='append', index=False, method='multi',
10         chunksize=1000)
11  logging.info("[LOAD] %s : %s lignes chargees", table_name,
12              len(df))
13
14  def charger_faits(df, engine):
15      """Truncate + bulk INSERT pour FAIT_VENTES."""
16      with engine.connect() as conn:
17          conn.execute(text(
18              "TRUNCATE TABLE dwh_mexora.fait_ventes RESTART
                IDENTITY CASCADE"
19          ))
20          conn.commit()
21  df.to_sql('fait_ventes', engine, schema='dwh_mexora',
22      if_exists='append', index=False, method='multi',
23      chunksize=5000)
24  logging.info("[LOAD] fait_ventes : %s lignes chargees",
25              len(df))

```

Listing 6 – loader.py — charger_dimension & charger_faits

2.8

```

1  def run_pipeline() -> dict:
2      # -- EXTRACT --
3      df_commandes_raw = extract_commandes(ACADEMIC_RAW_DIR /
4          'commandes_mexora.csv')
5      df_produits_raw = extract_produits (ACADEMIC_RAW_DIR /
6          'produits_mexora.json')
7      df_clients_raw = extract_clients (ACADEMIC_RAW_DIR /
8          'clients_mexora.csv')
9      df_regions = extract_regions (ACADEMIC_RAW_DIR /
10         'regions_maroc.csv')
11     # -- TRANSFORM --
12     df_commandes = transform_commandes(df_commandes_raw,
13         df_regions)
14     df_clients = transform_clients (df_clients_raw)

```

```

10     df_produits = transform_produits (df_produits_raw)
11     dim_temps   = build_dim_temps ('2020-01-01',
12                                   '2026-12-31') # 2 557 lignes
13     dim_client  = build_dim_client (df_clients, df_commandes,
14                                   df_regions) # 898
15     dim_produit = build_dim_produit(df_produits) # 300
16     dim_region  = build_dim_region (df_regions) # 12
17     dim_livreur = build_dim_livreur(df_commandes) # 6
18     fait_ventes = build_fait_ventes(df_commandes,
19                                   dim_temps, dim_client, dim_produit,
20                                   dim_region, dim_livreur) # 36 203
21     # -- LOAD (PostgreSQL) --
22     engine = get_pg_engine()
23     for name, df in [('dim_temps', dim_temps),
24                     ('dim_client', dim_client),
25                     ('dim_produit', dim_produit),
26                     ('dim_region', dim_region),
27                     ('dim_livreur', dim_livreur)]:
28         charger_dimension(df, name, engine)
29     charger_faits(fait_ventes, engine)
30     return {'fait_ventes': len(fait_ventes), 'duree_secondes':
31            14}

```

Listing 7 — main.py — orchestration complète E → T → L

Log de sortie — exécution validée (14 secondes)

```

[EXTRACT] Commandes : 50000 lignes
[TRANSFORM] R1 doublons      : 1500 lignes supprimees
[TRANSFORM] R5 quantites     : 390 lignes supprimees
[TRANSFORM] R6 prix nuls     : 1285 lignes supprimees
[TRANSFORM] R7 livreurs manq. : 6570 remplaces par -1
[TRANSFORM] Commandes       : 50000 -> 46825 lignes
[BUILD]    fait_ventes      : 36203 lignes produites
[LOAD]     dim_temps        : 2557 lignes chargees
[LOAD]     dim_client       : 898 lignes chargees
[LOAD]     dim_produit      : 300 lignes chargees
[LOAD]     fait_ventes      : 36203 lignes chargees
PIPELINE TERMINE EN 14 secondes

```

3 Étape 3 — Implémentation PostgreSQL (20 pts)

3.1

Le Data Warehouse utilise trois schémas PostgreSQL à responsabilités séparées :

```

1 CREATE SCHEMA IF NOT EXISTS staging_mexora;    -- zone brute /
   temporaire
2 CREATE SCHEMA IF NOT EXISTS dwh_mexora;        -- schema en
   etoile (dimensions + faits)
3 CREATE SCHEMA IF NOT EXISTS reporting_mexora;  -- vues
   materialisees analytiques

```

Listing 8 – Création des schémas PostgreSQL

3.2

```

1 CREATE TABLE dwh_mexora.dim_produit (
2     id_produit_sk SERIAL PRIMARY KEY,
3     id_produit_nk VARCHAR(20) NOT NULL,    -- cle naturelle
   source
4     nom_produit   VARCHAR(200) NOT NULL,
5     categorie     VARCHAR(100),
6     sous_categorie VARCHAR(100),
7     marque        VARCHAR(100),
8     fournisseur   VARCHAR(100),
9     prix_standard DECIMAL(10,2),
10    origine_pays   VARCHAR(50),
11    -- Colonnes SCD Type 2
12    date_debut     DATE NOT NULL DEFAULT CURRENT_DATE,
13    date_fin       DATE NOT NULL DEFAULT '9999-12-31',
14    est_actif      BOOLEAN NOT NULL DEFAULT TRUE,
15    CONSTRAINT uq_dim_produit_version UNIQUE (id_produit_nk,
   date_debut)
16 );

```

Listing 9 – DIM_PRODUT — avec colonnes SCD Type 2

```

1 CREATE TABLE dwh_mexora.fait_ventes (
2     id_vente      BIGSERIAL PRIMARY KEY,
3     id_commande   VARCHAR(30) NOT NULL,
4     id_date       INTEGER NOT NULL
5     REFERENCES dwh_mexora.dim_temps(id_date),
6     id_produit    INTEGER NOT NULL

```

```

7          REFERENCES
            dwh_mexora.dim_produit(id_produit_sk),
8  id_client  INTEGER      NOT NULL
9          REFERENCES dwh_mexora.dim_client(id_client_sk),
10 id_region  INTEGER      NOT NULL
11          REFERENCES dwh_mexora.dim_region(id_region),
12 id_livreur  INTEGER
13          REFERENCES dwh_mexora.dim_livreur(id_livreur),
14 -- Mesures additives
15 quantite_vendue  INTEGER      NOT NULL CHECK
16     (quantite_vendue > 0),
17 montant_ht       DECIMAL(12,2) NOT NULL,
18 montant_ttc      DECIMAL(12,2) NOT NULL, -- TVA 20 %
19 -- Mesure semi-additive
20 delai_livraison_jours SMALLINT,
21 -- Mesure non-additive
22 remise_pct       DECIMAL(5,2)  DEFAULT 0,
23 statut_commande  VARCHAR(20)
24     CHECK (statut_commande IN
            ('livre', 'annule', 'en_cours', 'retourne'))
25 );

```

Listing 10 – FAIT_VENTES — clés étrangères + contraintes CHECK

3.3

```

1  -- Index simples sur les cles etrangeres (jointures analytiques)
2  CREATE INDEX idx_fv_date      ON dwh_mexora.fait_ventes(id_date);
3  CREATE INDEX idx_fv_produit  ON
4      dwh_mexora.fait_ventes(id_produit);
5  CREATE INDEX idx_fv_client   ON
6      dwh_mexora.fait_ventes(id_client);
7  CREATE INDEX idx_fv_region   ON
8      dwh_mexora.fait_ventes(id_region);
9  CREATE INDEX idx_fv_livreur  ON
10     dwh_mexora.fait_ventes(id_livreur);
11
12 -- Index composite couvrant (requetes analytiques frequentes)
13 CREATE INDEX idx_fv_date_region
14     ON dwh_mexora.fait_ventes(id_date, id_region)
15     INCLUDE (montant_ttc, quantite_vendue);
16
17 -- Index partiel sur les commandes livrees (filtre dominant)
18 CREATE INDEX idx_fv_statut_livre
19     ON dwh_mexora.fait_ventes(statut_commande)
20     WHERE statut_commande = 'livre';

```

Listing 11 – Index simples et composites sur FAIT_VENTES
3.4**3.4.1**

```

1 CREATE MATERIALIZED VIEW reporting_mexora.mv_ca_mensuel AS
2 SELECT t.annee, t.mois, t.libelle_mois, t.periode_ramadan,
3        r.region_admin, r.zone_geo, p.categorie,
4        SUM(f.montant_ttc) AS ca_ttc,
5        COUNT(DISTINCT f.id_commande) AS nb_commandes,
6        SUM(f.quantite_vendue) AS volume_vendu,
7        ROUND(SUM(f.montant_ttc)
8              / NULLIF(COUNT(DISTINCT f.id_commande), 0), 2) AS
9              panier_moyen,
10        COUNT(DISTINCT f.id_client) AS nb_clients_actifs
11 FROM dwh_mexora.fait_ventes f
12 JOIN dwh_mexora.dim_temps t ON f.id_date = t.id_date
13 JOIN dwh_mexora.dim_region r ON f.id_region = r.id_region
14 JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produit_sk
15 WHERE f.statut_commande = 'livre'
16 GROUP BY t.annee, t.mois, t.libelle_mois, t.periode_ramadan,
17          r.region_admin, r.zone_geo, p.categorie
18 WITH DATA;
19 CREATE INDEX ON reporting_mexora.mv_ca_mensuel(annee, mois);
20 CREATE INDEX ON reporting_mexora.mv_ca_mensuel(region_admin);

```

Listing 12 – Vue matérialisée — CA mensuel par région et catégorie**3.4.2**

Classement par CA avec fonction fenêtre **RANK()** intra-catégorie et trimestre :

```

1 RANK() OVER (
2     PARTITION BY t.annee, t.trimestre, p.categorie
3     ORDER BY SUM(f.montant_ttc) DESC
4 ) AS rang_dans_categorie

```

Listing 13 – Vue matérialisée — Top produits par trimestre**3.4.3**

Taux de retard (délai > 3 jours) par livreur, zone et mois :

```

1 ROUND (
2     COUNT(*) FILTER (WHERE f.delai_livraison_jours > 3) * 100.0
3     / NULLIF(COUNT(*), 0), 2

```

```
4 ) AS taux_retard_pct
```

Listing 14 – Vue matérialisée — Taux de retard livreurs

3.5

```
1 -- Volume par table
2   objet      | lignes
3 -----+-----
4   dim_temps  |    2557
5   dim_produit |     300
6   dim_client |     898
7   dim_region |      12
8   dim_livreur |       6
9   fait_ventes |   36203
10
11 -- Contrôles d'intégrité référentielle
12   controle      | anomalies
13 -----+-----
14   faits_sans_date      |          0
15   faits_sans_produit   |          0
16   faits_sans_client    |          0
17   faits_sans_region    |          0
18   quantites_invalides |          0
19   montants_negatifs    |          0
```

Listing 15 – Résultats complets de check_integrity.sql

Intégrité référentielle : 0 anomalie sur 36 203 faits

Toutes les contraintes **CHECK** et **FOREIGN KEY** sont respectées. Aucune ligne orpheline, aucune quantité négative, aucun montant négatif.

4 Étape 4 — Dashboard de visualisation (15 pts)

4.1

4.1.1

```

1 WITH ca_mensuel AS (
2     SELECT annee, mois, region_admin, SUM(ca_ttc) AS ca_total
3     FROM reporting_mexora.mv_ca_mensuel
4     GROUP BY annee, mois, region_admin
5 )
6 SELECT region_admin,
7        ca_total
8        AS ca_actuel,
9        LAG(ca_total) OVER (PARTITION BY region_admin
10                           ORDER BY annee, mois) AS
11                           ca_precedent,
12        ROUND((ca_total - LAG(ca_total) OVER (PARTITION BY
13        region_admin ORDER BY annee, mois))
14        / NULLIF(LAG(ca_total) OVER (PARTITION BY
15        region_admin ORDER BY annee, mois), 0)*100,2) AS
16        evolution_pct
17 FROM ca_mensuel
18 ORDER BY annee DESC, mois DESC;
```

Listing 16 – KPI — CA mensuel avec évolution N-1

Résultat

Tanger **7 929 134 MAD** · Casablanca **7 793 512 MAD** · Rabat **6 035 814 MAD** ·
 Restant (9 villes) **23 826 835 MAD**

4.1.2

Filtrage sur `dim_region.ville = 'Tanger'` appliqué à `mv_top_produits` avec filtres interactifs trimestre et année (Metabase / Streamlit).

4.1.3

```

1 SELECT c.segment_client,
2        COUNT(DISTINCT f.id_vente) AS
3        nb_commandes,
4        ROUND(SUM(f.montant_ttc)
5        / COUNT(DISTINCT f.id_vente)::decimal, 2) AS
6        panier_moyen,
7        ROUND(SUM(f.montant_ttc) * 100.0
```

```

6          / SUM(SUM(f.montant_ttc)) OVER (), 2)          AS
          pct_ca
7 FROM dwh_mexora.fait_ventes f
8 JOIN dwh_mexora.dim_client c ON f.id_client = c.id_client_sk
9 WHERE f.statut_commande = 'livre' AND c.est_actif = TRUE
10 GROUP BY c.segment_client
11 ORDER BY panier_moyen DESC;

```

Listing 17 – Panier moyen et part de CA par segment

4.1.4

```

1 SELECT p.categorie,
2        COUNT(*) FILTER (WHERE f.statut_commande = 'retourne')
3        AS nb_retors,
4        COUNT(*)
5        AS nb_total,
6        ROUND(COUNT(*) FILTER (WHERE f.statut_commande =
7              'retourne')
8              * 100.0 / NULLIF(COUNT(*), 0), 2)          AS
9              taux_retour_pct
10 FROM dwh_mexora.fait_ventes f
11 JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produit_sk
12 GROUP BY p.categorie
13 ORDER BY taux_retour_pct DESC;

```

Listing 18 – Taux de retour — code couleur dans le dashboard

Résultat

Taux global **7,87 %** — **alerte rouge** activée sur toutes les catégories (seuil CDC : 5%).
Action prioritaire recommandée : audit de la politique de retour.

4.1.5

```

1 SELECT t.periode_ramadan,
2        ROUND(AVG(ca_ttc), 2) AS ca_moyen_mensuel,
3        SUM(volume_vendu)    AS volume_total
4 FROM reporting_mexora.mv_ca_mensuel
5 WHERE categorie = 'Alimentation'
6        AND annee >= 2022
7 GROUP BY t.periode_ramadan;

```

Listing 19 – Comparaison Ramadan vs hors Ramadan

Résultat

Indice de performance Ramadan = **91,2** (base 100 hors Ramadan). Les données synthétiques ne reproduisent pas l'effet saisonnier réel ; l'indicateur est conçu pour fonctionner avec des données transactionnelles réelles.

4.2

Metabase est conservé comme outil BI local conforme au cahier de charge (livrable L7). En complément, **Streamlit** a été ajouté comme interface web déployable pour faciliter la démonstration du projet en ligne.

Architecture Streamlit — découplage du Data Warehouse

- Les données sont exportées depuis PostgreSQL via `scripts/export_streamlit_data.py` vers `dashboard/data/*.csv`.
- L'application `dashboard/streamlit_app.py` lit uniquement ces CSV : **aucune connexion PostgreSQL n'est requise** sur Streamlit Cloud.
- Le dashboard couvre 5 pages : Vue générale, Analyse régionale, Clients, Produits, Retours & Ramadan.
- Lien déployé : <https://pipeline-etl-data-warehouse.streamlit.app/>

```
1 # 1. Exporter les donnees depuis PostgreSQL
2 python scripts/export_streamlit_data.py
3
4 # 2. Lancer localement (http://localhost:8501)
5 streamlit run dashboard/streamlit_app.py
6
7 # 3. Deployer sur Streamlit Cloud
8 #     -> share.streamlit.io / repo :
9 #         SoufianeZaari/Pipeline-ETL-Data-Warehouse
10 #     -> main file : dashboard/streamlit_app.py
```

Listing 20 — Workflow complet — export et déploiement Streamlit

4.3

Indicateur	Insight et recommandation
CA total	45 585 295 MAD sur le périmètre chargé
Top région	Tanger +1,7 % vs Casablanca — consolider le marché local tangérois
Top catégorie	Electronics 82 % du CA — sécuriser les stocks et les fournisseurs
Taux de retour	7,87 % (seuil : 5 %) — auditer la politique retour et les fiches produit
Délai livraison	14,99 jours en moyenne — objectif < 7 jours via SLA transporteurs
Segment Gold	Panier moyen le plus élevé — programme fidélité VIP à prioriser

Table 7 — Insights métier identifiés dans le Data Warehouse Mexora

5 Conclusion

Ce mini-projet démontre la mise en œuvre complète d'un système décisionnel d'entreprise, de l'analyse des besoins jusqu'au dashboard déployé, en respectant rigoureusement les quatre étapes du cahier de charge.

Bilan technique par étape

Étape 1	Modélisation en étoile rigoureuse (5 dimensions), granularité justifiée, structure SCD Type2 préparée via <code>date_debut</code> , <code>date_fin</code> , <code>est_actif</code> sur DIM_PRODUT et DIM_CLIENT (historisation multi-version activable dans les évolutions futures du DWH).
Étape 2	Pipeline Python structuré <code>mexora_etl/</code> , 14 règles de transformation documentées et loguées, segmentation client automatisée, 14 secondes d'exécution pour 50 000 lignes source.
Étape 3	PostgreSQL 16 avec 3 schémas dédiés, 36 203 faits chargés, 3 vues matérialisées fonctionnelles, 0 anomalie sur 6 contrôles d'intégrité référentielle.
Étape 4	Metabase comme dashboard BI local conforme au cahier de charge, 5 questions analytiques avec KPIs et alertes visuelles. Complément : application Streamlit déployée en ligne, autonome via des exports CSV du Data Warehouse.

1. **TVA 20 %** appliquée uniformément (`montant_ttc = montant_ht × 1.20`) en l'absence de taux différenciés par catégorie dans les données source.
2. **Livreurs fictifs** (L001–L005 + inconnu –1) enrichis avec des métadonnées statiques, faute de référentiel transporteur dans les sources.
3. **Statuts inconnus** exclus de la table de faits (contrainte `CHECK` PostgreSQL) et tracés dans les logs ETL.
4. **Instance PostgreSQL locale** sur le port 5433 (socket `/tmp/mexora_pg_socket`) pour contournement des droits administrateur sur la machine académique.
5. **Segmentation Streamlit** recalculée par percentiles (P30/P70) car le CA synthétique par client dépasse largement les seuils fixes (67 K–668 K MAD), rendant la classification fixe mono-segment. Le schéma DWH n'est pas modifié.